

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **ИНФОРМАЦИОННЫЕ СИСТЕМЫ И ТЕЛЕКОММУНИКАЦИИ (ИУ3)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.03 ПРИКЛАДНАЯ ИНФОРМАТИКА**

ОТЧЕТ

по лабораторной работе № 3

Название: Подготовка обучающей и тестовой выборки, кросс-валидация и подбор гиперпараметров на примере метода ближайших соседей

Дисциплина: Технологии машинного обучения

Студент	ИУ3С-51Б		М.Ю. Гордон
	(Группа)	(Подпись, дата)	(И.О. Фамилия)
Преподаватель			О.Г. Саргсян
		(Подпись, дата)	(И.О. Фамилия)

Москва, 2026

1. Описание задания

Цель лабораторной работы — изучение способов подготовки выборки и подбора гипермараметров на примере метода ближайших соседей.

- Выбрать набор данных (датасет), для решения задачи классификации или регрессии.
- В случае необходимости провести удаление или заполнение пропусков и кодирование категориальных признаков.
- С использованием метода train_test_split разделить выборку обучающую и тестовую.
- Обучить модель ближайших соседей для произвольно заданного гипермараметра К. Оценить качество модели с помощью подходящих для задачи метрик.
- Произвести подбор гипермараметра К с использованием GridSearchCV и RandomixedSearchCV и кросс-валидации, оценить качество оптимальной модели. Использовать не менее двух стратегий кросс-валидации.
- Сравнить метрики качества исходной и оптимальной моделей.

2. Текст программы по ноутбукам и их результаты

```
import pandas as pd

df = pd.read_csv("penguins_size.csv")

print("Размер датасета:", df.shape)
print("\nПервые 5 строк:")
display(df.head())

print("\nНазвания столбцов:")
print(df.columns.tolist())

print("\nПропуски:")
print(df.isnull().sum())
```

Размер датасета: (344, 7)

Первые 5 строк:

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	MALE
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	FEMALE
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	FEMALE
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	FEMALE

Названия столбцов:
['species', 'island', 'culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g', 'sex']

Пропуски:
species 0
island 0
culmen_length_mm 2
culmen_depth_mm 2
flipper_length_mm 2
body_mass_g 2
sex 10
dtype: int64

Рисунок 1 - результат 1 ноутбука

```
# Типы данных в каждом столбце
print("Типы данных:")
print(df.dtypes)

# Количество объектов каждого вида
print("\nКоличество пингвинов каждого вида:")
print(df["species"].value_counts())

# Основная информация о датасете
print("\nОбщая информация:")
df.info()
```

Типы данных:
species str
island str
culmen_length_mm float64
culmen_depth_mm float64
flipper_length_mm float64
body_mass_g float64
sex str
dtype: object

Рисунок 2 - Типы данных датасета

Количество пингвинов каждого вида:
species
Adelie 152
Gentoo 124
Chinstrap 68
Name: count, dtype: int64

Рисунок 3 - Количество пингвинов каждого вида

```
Общая информация:
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   culmen_length_mm   342 non-null    float64
 3   culmen_depth_mm    342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                334 non-null    str
```

Рисунок 4 - Общая информация по датасету

```
# Количество пропусков
print("Пропуски по столбцам:")
print(df.isnull().sum())

# Уникальные значения категориальных столбцов
print("\nВиды пингвинов:")
print(df["species"].unique())

print("\nОстрова:")
print(df["island"].unique())

print("\nЗначения пола:")
print(df["sex"].unique())
```

```
Пропуски по столбцам:
species              0
island               0
culmen_length_mm     2
culmen_depth_mm      2
flipper_length_mm    2
body_mass_g          2
sex                 10
dtype: int64
```

Рисунок 5 - Пропуски по столбцам

```
Виды пингвинов:
<StringArray>
['Adelie', 'Chinstrap', 'Gentoo']
Length: 3, dtype: str

Острова:
<StringArray>
['Torgersen', 'Biscoe', 'Dream']
Length: 3, dtype: str

Значения пола:
<StringArray>
['MALE', 'FEMALE', nan, '.']
```

Рисунок 6 - Уникальные значения категориальных столбцов

```
# Создаём копию исходного датасета
df_clean = df.copy()

# Заменяем точку в столбце sex на пропуск
df_clean["sex"] = df_clean["sex"].replace(".", pd.NA)

# Удаляем столбец sex
```

```
df_clean = df_clean.drop(columns=["sex"])

# Удаляем строки с пропусками в числовых признаках
df_clean = df_clean.dropna()

print("Размер исходного датасета:", df.shape)
print("Размер после обработки:", df_clean.shape)

print("\nПропуски после обработки:")
print(df_clean.isnull().sum())

display(df_clean.head())
```

```
Размер исходного датасета: (344, 7)
Размер после обработки: (342, 6)

Пропуски после обработки:
species              0
island               0
culmen_length_mm     0
culmen_depth_mm      0
flipper_length_mm    0
body_mass_g          0
dtype: int64
```

	species	island	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0

Рисунок 7 - Результат обработки пропусков

```
# Кодируем категориальный признак island
df_encoded = pd.get_dummies(
    df_clean,
    columns=["island"],
    dtype=int
)

print("Столбцы после кодирования:")
print(df_encoded.columns.tolist())

display(df_encoded.head())
```

```
Столбцы после кодирования:
['species', 'culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g', 'island_Biscoe', 'island_Dream', 'island_Torgersen']
```

	species	culmen_length_mm	culmen_depth_mm	flipper_length_mm	body_mass_g	island_Biscoe	island_Dream	island_Torgersen
0	Adelie	39.1	18.7	181.0	3750.0	0	0	1
1	Adelie	39.5	17.4	186.0	3800.0	0	0	1
2	Adelie	40.3	18.0	195.0	3250.0	0	0	1
4	Adelie	36.7	19.3	193.0	3450.0	0	0	1
5	Adelie	39.3	20.6	190.0	3650.0	0	0	1

Рисунок 8 - Результат кодирования категориального признака

```
# Признаки
```

```
X = df_encoded.drop(columns=["species"])

# Целевая переменная
y = df_encoded["species"]

print("Размер X:", X.shape)
print("Размер y:", y.shape)

print("\nПризнаки модели:")
print(X.columns.tolist())

print("\nЦелевые классы:")
print(y.unique())
```

```
Размер X: (342, 7)
Размер y: (342,)

Признаки модели:
['culmen_length_mm', 'culmen_depth_mm', 'flipper_length_mm', 'body_mass_g', 'island_Biscoe', 'island_Dream', 'island_Torgersen']

Целевые классы:
<StringArray>
['Adelie', 'Chinstrap', 'Gentoo']
Length: 3, dtype: str
```

Рисунок 9 - Результат 6 ноутбука

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
   X,
   y,
   test_size=0.2,
   random_state=42,
   stratify=y
)

print("Обучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)

print("\nВиды пингвинов в обучающей выборке:")
print(y_train.value_counts())

print("\nВиды пингвинов в тестовой выборке:")
print(y_test.value_counts())
```

```
Обучающая выборка: (273, 7)
Тестовая выборка: (69, 7)
```

Рисунок 10 - Количество строк обучающей и тестовой выборок

```
Виды пингвинов в обучающей выборке:
species
Adelie      121
Gentoo       98
Chinstrap    54
Name: count, dtype: int64

Виды пингвинов в тестовой выборке:
species
Adelie      30
Gentoo      25
Chinstrap   14
Name: count, dtype: int64
```

Рисунок 11 - Виды пингвинов в обучающей и тестовой выборках

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report

# Создаём модель с K = 5
knn_initial = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])

# Обучаем модель
knn_initial.fit(X_train, y_train)

# Получаем предсказания
y_pred_initial = knn_initial.predict(X_test)

# Оцениваем качество
initial_accuracy = accuracy_score(y_test, y_pred_initial)

print("Начальная модель: K = 5")
print("Accuracy:", round(initial_accuracy, 4))

print("\nОтчёт о классификации:")
print(classification_report(y_test, y_pred_initial))
```

```
Начальная модель: K = 5
Accuracy: 1.0

Отчёт о классификации:
              precision    recall  f1-score   support

      Adelie       1.00      1.00      1.00        30
   Chinstrap       1.00      1.00      1.00        14
      Gentoo       1.00      1.00      1.00        25

    accuracy                           1.00        69
   macro avg       1.00      1.00      1.00        69
weighted avg       1.00      1.00      1.00        69
```

Рисунок 12 - Результат обучения модели методу ближайших соседей

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

# Стратегия кросс-валидации № 1
cv_stratified = StratifiedKFold(
```

```
    n_splits=5,
    shuffle=True,
    random_state=42
)

# Значения K, которые проверим
param_grid = {
    "knn__n_neighbors": list(range(1, 31))
}

# Настраиваем GridSearchCV
grid_search = GridSearchCV(
    estimator=Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier())
    ]),
    param_grid=param_grid,
    scoring="accuracy",
    cv=cv_stratified,
    n_jobs=-1
)

# Подбираем K только на обучающей выборке
grid_search.fit(X_train, y_train)

print("Лучшее значение K:", grid_search.best_params_["knn__n_neighbors"])
print("Лучшая средняя Accuracy на кросс-валидации:",
      round(grid_search.best_score_, 4))

# Проверяем лучшую модель на тестовой выборке
best_grid_model = grid_search.best_estimator_
y_pred_grid = best_grid_model.predict(X_test)

grid_accuracy = accuracy_score(y_test, y_pred_grid)

print("\nAccuracy на тестовой выборке:",
      round(grid_accuracy, 4))

print("\nОтчёт о классификации:")
print(classification_report(y_test, y_pred_grid))
```

```
Лучшее значение K: 3
Лучшая средняя Accuracy на кросс-валидации: 1.0

Accuracy на тестовой выборке: 1.0
```

Рисунок 13 - Результат выбора лучшего K из 30

```
Отчёт о классификации:
              precision    recall  f1-score   support

      Adelie       1.00      1.00      1.00        30
   Chinstrap       1.00      1.00      1.00        14
      Gentoo       1.00      1.00      1.00        25

    accuracy                           1.00        69
   macro avg       1.00      1.00      1.00        69
weighted avg       1.00      1.00      1.00        69
```

Рисунок 14 - Отчет о классификации

```
from sklearn.model_selection import RandomizedSearchCV, RepeatedStratifiedKFold

# Стратегия кросс-валидации № 2
cv_repeated = RepeatedStratifiedKFold(
   n_splits=5,
   n_repeats=2,
   random_state=42
)

# Диапазон значений K для случайного поиска
param_distributions = {
   "knn__n_neighbors": list(range(1, 31))
}

random_search = RandomizedSearchCV(
   estimator=Pipeline([
       ("scaler", StandardScaler()),
       ("knn", KNeighborsClassifier())
   ]),
   param_distributions=param_distributions,
   n_iter=15,
   scoring="accuracy",
   cv=cv_repeated,
   random_state=42,
   n_jobs=-1
)

# Подбор параметра на обучающей выборке
random_search.fit(X_train, y_train)

print("Лучшее значение K:",
     random_search.best_params_["knn__n_neighbors"])

print("Лучшая средняя Accuracy на кросс-валидации:",
     round(random_search.best_score_, 4))

# Проверка оптимальной модели на тестовой выборке
best_random_model = random_search.best_estimator_
y_pred_random = best_random_model.predict(X_test)

random_accuracy = accuracy_score(y_test, y_pred_random)

print("\nAccuracy на тестовой выборке:",
     round(random_accuracy, 4))

print("\nОтчёт о классификации:")
print(classification_report(y_test, y_pred_random))
```

```
Лучшее значение K: 13
Лучшая средняя Accuracy на кросс-валидации: 0.9908

Accuracy на тестовой выборке: 1.0
```

Рисунок 15 - Результат выбора лучшего K из 15 случайных вариантов

```
Отчёт о классификации:
              precision    recall  f1-score   support

      Adelie       1.00      1.00      1.00        30
   Chinstrap       1.00      1.00      1.00        14
      Gentoo       1.00      1.00      1.00        25

    accuracy                           1.00        69
   macro avg       1.00      1.00      1.00        69
weighted avg       1.00      1.00      1.00        69
```

Рисунок 16 - Отчет о классификации

```
comparison = pd.DataFrame({
    "Модель": [
        "Исходная KNN",
        "GridSearchCV",
        "RandomizedSearchCV"
    ],
    "Оптимальное K": [
        5,
        grid_search.best_params_["knn__n_neighbors"],
        random_search.best_params_["knn__n_neighbors"]
    ],
    "Accuracy на тесте": [
        initial_accuracy,
        grid_accuracy,
        random_accuracy
    ],
    "Средняя Accuracy CV": [
        None,
        grid_search.best_score_,
        random_search.best_score_
    ]
})

display(comparison.round(4))
```

	Модель	Оптимальное K	Accuracy на тесте	Средняя Accuracy CV
0	Исходная KNN	5	1.0	NaN
1	GridSearchCV	3	1.0	1.0000
2	RandomizedSearchCV	13	1.0	0.9908

Рисунок 17 - Таблица сравнения моделей

3. Вывод

В ходе лабораторной работы были изучены подготовка обучающей и тестовой выборок, кросс-валидация и подбор гиперпараметров метода ближайших соседей.

Для классификации был выбран датасет видов пингвинов, выполнена

обработка пропущенных значений и кодирование категориальных признаков. Данные разделила на обучающую и тестовую выборки в соотношении 80:20.

В Для исходной модели использовалось K = 5, а с помощью GridSearchCV и RandomizedSearchCV были подобраны значения K = 3 и K = 13. Таким образом, подбор гиперпараметров позволил сравнить различные настройки модели, но не повысил точность по сравнению с исходным вариантом.